AMCJワークショップ
共鳴するコード、SuperColliderで創る音の世界 - 実践とその先へ

01.
準備体操: SuperColliderのおさらい
- Sin波からはじめるサウンドプログラミング

2026年3月14日

田所淳

01: SuperColliderの基本

完成イメージ

```
SynthDef("mySynth", {
  arg out = 0, gate = 1, freq = 440, detune = 1.005,
      cutoff = 1200, cutfreq = 1.0, amp = 0.2;
  var sig, cutsig, env;
  sig = Mix.ar([
    Saw.ar([freq / detune, freq * detune]),
    Saw.ar([freq / detune, freq * detune] / detune),
    Saw.ar([freq / detune, freq * detune] / detune / 2.0)
  ]) / 3.0;
  env = EnvGen.kr(Env.adsr(), gate, doneAction: 2);
  sig = sig * env * amp;
  cutsig = LFNoise2.kr([cutfreq, cutfreq * 1.5]).range(cutoff * 0.25, cutoff * 1.5);
  sig = MoogFF.ar(sig, cutsig, mul: 2.0);
  sig = GVerb.ar(sig, 100, 1.0);
  Out.ar(out, sig);
}).add;
```

ステップバイステップでこれを作っていきます

このセクションの内容

1. 文法の基本
2. 音響を生成
3. 楽器（SynthDef）の作成
4. パターンで演奏

1. 文法の基本

Hello World — オブジェクトとメソッド

```
// オブジェクト.メソッド
"Hello world".postln;

// 別の書き方（関数形式）
postln("Hello world");

// メソッドの連結
"Hello world".reverse.postln;
```

SuperColliderでは**すべてがオブジェクト**。
`.メソッド名` で操作を連鎖できる。

変数 — var と環境変数

```
// ローカル変数: まとめて実行 (Ctrl+Enter)
(
var str;
str = "Hello world";
str.postln;
)

// 環境変数: 1行ずつ実行も可能
~str = "Hello world";
~str.postln;

// 1文字の変数は環境変数として使える
a = "Hello world";
a.postln;
```

`~変数名` はどこからでもアクセスできる**環境変数**。

関数の定義と実行

```
// 関数を定義して環境変数に代入
(
  ~func = {
    var msg1, msg2;
    msg1 = "Hello";
    msg2 = "world";
    (msg1 + msg2).postln;
  }
)

// 関数を実行
~func.value;
```

`{ ... }` が**関数（Function）**。 `.value` で実行する。

関数への引数

```
// 引数を持つ関数
(
  ~func = {
    arg msg1, msg2;
    var bindMsg;
    bindMsg = msg1 + msg2;
    bindMsg.postln;
  }
)

// 引数を指定して実行
~func.value("Hello", "World");
~func.value("こんにちは", "世界");
```

`arg` キーワードで**引数**を定義する。

2. 音響を生成

サーバーの起動と停止

```
// SuperColliderオーディオサーバーを起動
s.boot;

// すべてのサーバーを停止
Server.killAll;
```

`s` はデフォルトの**オーディオサーバー（scsynth）**。
音を出す前に必ず起動が必要。

最初の音 — SinOsc

```
// サイン波を再生する関数を定義
~sine = {SinOsc.ar()};
// 関数を実行して音を鳴らす
~sine.play;

// 1行で書くこともできる
{SinOsc.ar()}.play;

// ステレオで再生
{SinOsc.ar().dup(2)}.play;
```

- `SinOsc.ar()` — サイン波を生成するUGen
- `.dup(2)` — 同じ信号を2チャンネルに複製（ステレオ）
- `Cmd+.` で音を止める

周波数と音量の指定

```
// 周波数を指定 SinOsc.ar(freq, phase, mul, add)
{SinOsc.ar(110).dup(2)}.play;

// 音量(mul)を指定
{SinOsc.ar(110, 0, 0.8).dup(2)}.play;

// キーワード引数で指定
{SinOsc.ar(freq:110, mul:0.8).dup(2)}.play;
```

引数はキーワード形式（`名前:値`）でも指定できる。

変数を使って整理する

```
(
~func = {
    var freq = 440, amp = 0.8;
    SinOsc.ar(freq, 0, amp).dup(2);
  }
)
~func.play;
```

変数に名前をつけることで**コードが読みやすく**なる。

3. 楽器（SynthDef）の作成

SynthDefの基本

```
// .playより厳密な方法 — SynthDefを定義
(
  SynthDef("mySynth", {
    arg out = 0, freq = 440, amp = 0.8;
    var sig;
    sig = SinOsc.ar(freq) * amp;
    Out.ar(out, sig.dup(2));
  }).add;
)
```

- `SynthDef("名前", { ... })` — 楽器の設計図を定義
- `arg` — 外部から変更できるパラメータ
- `Out.ar(out, sig)` — オーディオ出力

SynthDefを演奏する

```
// Synthを生成（音が鳴り始める）
a = Synth("mySynth");

// パラメータをリアルタイムで変更
a.set(freq: 330, amp: 0.5);
a.set(freq: 110, amp: 0.5);

// 音を停止して解放
a.free;

// 引数を指定して生成
b = Synth("mySynth", [freq: 880, amp: 0.8]);
b.free;
```

SinOsc → Saw波に変更

```
(
  SynthDef("mySynth", {
    arg out = 0, freq = 440, amp = 0.8;
    var sig;
    sig = Saw.ar(freq) * amp;  // SinOsc → Saw
    Out.ar(out, sig.dup(2));
  }).add;
)
a = Synth("mySynth", [freq: 110]);
a.free;
```

`Saw.ar()` — のこぎり波UGen。倍音が豊富でより存在感のある音。

デチューンで厚みを出す

```
(
  SynthDef("mySynth", {
    arg out = 0, freq = 440, detune = 1.005, amp = 0.8;
    var sig;
    sig = Saw.ar([freq / detune, freq * detune], amp);
    Out.ar(out, sig);
  }).add;
)
a = Synth("mySynth", [freq: 110, detune: 1.005, amp: 0.8]);
a.free;
```

周波数をわずかにずらして2つ重ねることで**厚みのあるサウンド**に。

Mix.arで複数の波を重ねる

```
(
  SynthDef("mySynth", {
    arg out = 0, freq = 440, detune = 1.005, amp = 0.8;
    var sig;
    sig = Mix.ar([
      Saw.ar([freq / detune, freq * detune]),
      Saw.ar([freq / detune, freq * detune] / detune),
      Saw.ar([freq / detune, freq * detune] / detune / 2.0)
    ]) / 3.0 * amp;
    Out.ar(out, sig);
  }).add;
)
a = Synth("mySynth", [freq: 110, detune: 1.005, amp: 0.8]);
a.free;
```

`Mix.ar([...])` — 複数の信号を**ミックス**する。`/ 3.0` で音量を調整。

エンベロープを追加

```
(
  SynthDef("mySynth", {
    arg out = 0, gate = 1, freq = 440, detune = 1.005, amp = 0.8;
    var sig, env;
    sig = Mix.ar([
      Saw.ar([freq / detune, freq * detune]),
      Saw.ar([freq / detune, freq * detune] / detune),
      Saw.ar([freq / detune, freq * detune] / detune / 2.0)
    ]) / 3.0;
    env = EnvGen.kr(Env.adsr(), gate, doneAction: 2);
    sig = sig * env * amp;
    Out.ar(out, sig);
  }).add;
)
a = Synth("mySynth", [\freq, 110]);
a.set(\gate, 0);  // gate=0で音が止まる
```

`Env.adsr()` — Attack, Decay, Sustain, Releaseエンベロープ。

フィルターを追加

```
(
  SynthDef("mySynth", {
    arg out = 0, gate = 1, freq = 440, detune = 1.005,
        cutoff = 2000, amp = 0.8;
    var sig, env;
    sig = Mix.ar([
      Saw.ar([freq / detune, freq * detune]),
      Saw.ar([freq / detune, freq * detune] / detune),
      Saw.ar([freq / detune, freq * detune] / detune / 2.0)
    ]) / 3.0;
    env = EnvGen.kr(Env.adsr(), gate, doneAction: 2);
    sig = sig * env * amp;
    sig = MoogFF.ar(sig, cutoff, mul: 2.0);
    Out.ar(out, sig);
  }).add;
)
a = Synth("mySynth", [\freq, 110]);
a.set(\gate, 0);
```

`MoogFF` — Moogスタイルのローパスフィルター。

フィルターの周波数を揺らす

```
(
  SynthDef("mySynth", {
    arg out = 0, gate = 1, freq = 440, detune = 1.005,
        cutoff = 1200, cutfreq = 1.0, amp = 0.8;
    var sig, cutsig, env;
    sig = Mix.ar([
      Saw.ar([freq / detune, freq * detune]),
      Saw.ar([freq / detune, freq * detune] / detune),
      Saw.ar([freq / detune, freq * detune] / detune / 2.0)
    ]) / 3.0;
    env = EnvGen.kr(Env.adsr(), gate, doneAction: 2);
    sig = sig * env * amp;
    cutsig = LFNoise2.kr([cutfreq, cutfreq * 1.5]).range(cutoff * 0.25, cutoff * 1.5);
    sig = MoogFF.ar(sig, cutsig, mul: 2.0);
    Out.ar(out, sig);
  }).add;
)
a = Synth("mySynth", [\freq, 110]);  a.set(\gate, 0);
```

`LFNoise2.kr` — ランダムな低周波ノイズでフィルター周波数を変化させる。

リバーブを追加して完成！

```
(
SynthDef("mySynth", {
  arg out = 0, gate = 1, freq = 440, detune = 1.005,
      cutoff = 1200, cutfreq = 1.0, amp = 0.8;
  var sig, cutsig, env;
  sig = Mix.ar([
    Saw.ar([freq / detune, freq * detune]),
    Saw.ar([freq / detune, freq * detune] / detune),
    Saw.ar([freq / detune, freq * detune] / detune / 2.0)
  ]) / 3.0;
  env = EnvGen.kr(Env.adsr(), gate, doneAction: 2);
  sig = sig * env * amp;
  cutsig = LFNoise2.kr([cutfreq, cutfreq * 1.5]).range(cutoff * 0.25, cutoff * 1.5);
  sig = MoogFF.ar(sig, cutsig, mul: 2.0);
  sig = GVerb.ar(sig, 100, 1.0);  // リバーブ追加
  Out.ar(out, sig);
}).add;
)
a = Synth("mySynth", [\freq, 110]);  a.set(\gate, 0);
```

`GVerb` — ホール系リバーブ。空間の広がりが加わって完成！

4. パターンで演奏する

Pbindで自動演奏

```
(
TempoClock.default.tempo = 80/60;
p = Pbind(
    \instrument, "mySynth",
    \scale,      Scale.indian(\pythagorean),
    \degree,     Prand([0, 1, 2, 3, 4], inf),
    \octave,     Prand([2, 3, 3, 4, 5], inf),
    \detune,     Pwhite(1.0, 1.005),
    \cutoff,     Pwhite(100, 4000),
    \amp,        Pwhite(0.15, 0.5),
    \dur,        Prand([1/4, 1/2, 1/2, 1], inf),
    \legato,     Pwhite(0.5, 1.0)
).play;
)
p.stop;
```

`Pbind` — SynthDefをパターンで自動演奏。`Prand` / `Pwhite` でランダム変化。

複数のパターンを同時に演奏

```
(
TempoClock.default.tempo = 80/60;
~p1 = Pbind(
    \instrument, "mySynth",
    \scale,   Scale.indian(\pythagorean),
    \degree,  Prand([0, 1, 2, 3, 4], inf),
    \octave,  Prand([2, 3, 3, 4, 5], inf),
    \detune,  Pwhite(1.0, 1.005),
    \cutoff,  Pwhite(100, 4000),
    \amp,     Pwhite(0.15, 0.5),
    \dur,     Prand([1/4, 1/2, 1/2, 1], inf),
    \legato,  Pwhite(0.5, 1.0)
).play;
~p2 = Pbind(
    \instrument, "mySynth",
    \scale,   Scale.indian(\pythagorean),
    \degree,  Prand([0, 1, 2, 3, 4], inf),
    \octave,  Prand([4, 5, 5, 6], inf),
    \detune,  Pwhite(1.0, 1.005),
    \cutoff,  Pwhite(100, 4000),
    \amp,     Pwhite(0.15, 0.5),
    \dur,     Prand([1/4, 1/4, 1/2], inf),
    \legato,  Pwhite(0.5, 1.0)
).play;
)
~p1.stop; ~p2.stop;
```

複数の `Pbind` を環境変数に代入し、個別に停止できる。

まとめ

セクション	内容
文法の基本	オブジェクト・メソッド・変数・関数
音響を生成	SinOsc / Saw / .play / .dup(2)
SynthDef	引数・Mix・EnvGen・MoogFF・GVerb
パターン	Pbind / Prand / Pwhite / Scale

SynthDefの構成要素:

- `Saw` — 波形 → `Mix` — 重ねる → `EnvGen` — 音量形状
- `MoogFF` — フィルター → `GVerb` — 空間

次のセッション: SuperCollider + VSCodeでVibe Coding！